

基于 K1 MUSE Pi Pro 的 AI 智能药盒与健康问诊系统

目录

摘要.....	2
第一部分 作品概述.....	3
1.1 功能与特性.....	3
1.2 应用领域.....	4
1.3 主要技术特点.....	4
1.4 主要性能指标.....	5
1.5 主要创新点.....	5
1.6 设计流程.....	6
第二部分 系统组成及功能说明.....	6
2.1 整体介绍.....	6
2.2 硬件系统介绍.....	7
2.3 机械设计介绍.....	7
2.3.1 电路各模块介绍.....	8
2.4 软件系统介绍.....	11
2.4.1 软件整体介绍.....	11
2.4.2 软件各模块介绍.....	11
第三部分 完成情况及性能参数.....	14
3.1 整体介绍.....	14
3.2 工程成果.....	15
3.2.1 机械成果.....	15
3.2.2 电路成果.....	15
3.2.3 软件成果.....	16
第四部分 总结.....	17
4.1 可扩展之处.....	17
4.2 心得体会.....	18
第五部分 参考文献.....	18

摘要

本项目设计并实现了一款面向居家养老与智慧医疗场景的智能药盒辅助系统。系统以 MUSE Pi Pro 为主控平台，融合了语音识别 (ASR)、大语言模型 (LLM) 与物联网硬件控制技术。用户无需触控，仅需通过麦克风描述身体症状或说出药品名称，系统即可调用百度语音识别引擎将音频转为文字，并借助阿里云通义千问大模型 (Qwen-turbo) 进行智能问诊分析或药品百科查询，最终将诊断结果通过语音合成 (TTS) 进行拟人化播报。在硬件执行端，系统集成 DHT11 温湿度传感器以实现居家环境监测，并通过 SG90 舵机驱动盒盖开合，配合 5 路 LED 指示灯进行精准定位，实现“听声识病、自动配药、灯光指引”的自动化流程。该系统有效解决了老年人及视力障碍群体在用药过程中操作复杂、识别困难等问题，具有较强的人机交互友好性与实用推广价值。

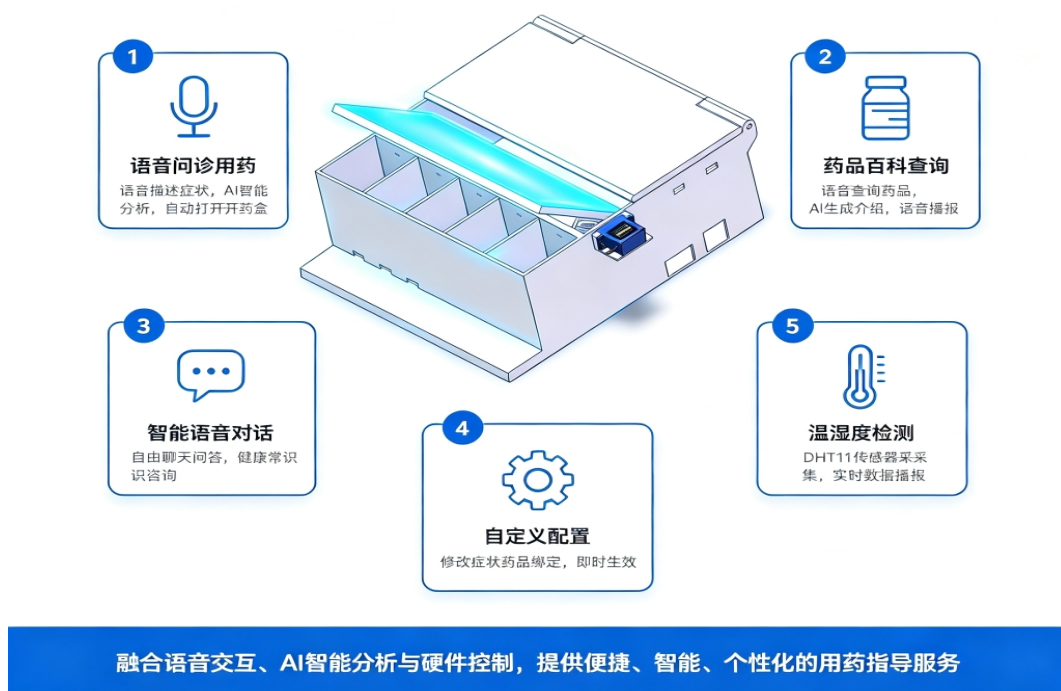


图1-1 系统核心能力示意图

第一部分 作品概述

1.1 功能与特性

本作品是一款集“智能语音交互”与“物理药盒控制”为一体的嵌入式辅助设备，主要具备以下核心功能特性：

1.语音交互问诊与用药：用户可通过口述身体不适症状，系统经云端的 AI 模型分析后，输出包含药品名称、服用剂量、每日次数及注意事项的完整用药方案。

2.药品智能语音检索：用户可语音查询特定药品的功效、用法及禁忌，系统通过 AI 生成口语化回复并进行语音播报，充当家庭药师角色。

3.实体药盒自动联动：系统内部将常见疾病与对应药品进行“映射绑定”。当 AI 分析出的推荐药品与系统预设匹配时，系统会自动驱动舵机打开药盒顶盖，并点亮对应药格位置的 LED 指示灯，辅助用户精准取药。

4.环境状态感知：集成 DHT11 传感器，支持实时监测并语音播报当前环境温度与湿度，辅助药品储存环境管理。

动态自配置：支持在终端菜单中随时修改“症状”与“药品”的映射关系，无需修改底层代码，灵活适配不同家庭的用药习惯。

1.2 应用领域



图 1.1 应用场景示意图

本项目主要面向智慧医疗、居家养老及家庭健康管理领域。在我国老龄化进程加速的背景下，独居老人及慢性病患者经常面临“忘记吃药”、“吃错药”或者“看不懂说明书”的痛点。本系统将 AI 大模型引入日常用药场景，将原本复杂的药品说明书和疾病自查过程，转化为简单的“开口说话”交互方式。它非常适合用于：

- 家庭药箱：为家庭成员提供日常轻症（如腹泻、关节痛）的初步用药指导。
- 社区养老服务中心：作为辅助医疗设备，帮助护理人员快速为老人提供用药提醒与常识解答。
- 视障人士辅助：完全基于语音的交互逻辑，使视力障碍人群也能独立安全地完成取药与用药咨询。

1.3 主要技术特点

端云协同的混合式 AI 架构：巧妙将低成本的本地硬件控制（MUSE Pi Pro）与高算力的云端大语言模型（Qwen-turbo）及语音处理（百度 API）相结合，既保证了嵌入式系统的实时控制，又赋予了设备强大的自然语言理解与生成能力。

映射驱动的硬件动作引擎：系统在软件设计上使用了“数据驱动”的思想。通

过一个可动态配置的 bind_list（症状->药品->LED->舵机）字典列表，实现了大模型结果、数字逻辑判断与物理硬件执行的完美解耦。

无抖动平滑电机控制算法：针对舵机在启停时易产生机械抖动的问题，系统代码中设计了缓动（Smooth Turn）控制算法，通过细分步进角度和微秒级延时，实现了药盒盖顺滑开合，提升了机械结构的稳定性与用户体验。

4. 纯语音交互闭环：形成了“用户说 -> 机器听 -> 云端算 -> 机器做 -> 机器舵机精度说”的完整无接触交互闭环，极大地降低了学习门槛。

1.4主要性能指标

性能类别	具体指标参数
主控平台	MUSE Pi Pro
音频采样率	16kHz / 16bit 单声道
语音引擎	百度语音识别（ASR）+ 语音合成（TTS）
AI 大模型	阿里云通义千问（Qwen-turbo）
舵机精度	SG90, 0~180°可调，平滑步进角度 2°
环境检测	支持 DHT11，检测温湿度，读取失败自动重试 5次
交互延时	约 3~5 秒（依赖家庭网络带宽）

1.5主要创新点

“对症开盒”的智能物联网药盒：区别于传统仅具备定时提醒功能的药盒，本系统首次将 AI 大模型的文本语义理解转化为物理空间的“精准开盒+指引亮灯”动作，实现了药盒的主动式服务。

软硬件高度解耦的设计思想：设计了一种基于列表映射的动态绑定机制，最终用户（或社区医生）可以通过简单的命令行修改“疾病-药品”映射表，无需具备编程能力即可定制药盒功能。

低成本原型验证方案：采用废弃纸盒与面包板搭建原型，结合 MUSE Pi Pro 与开源 API，以极低硬件成本实现了高附加值的功能，证明了该方案的可行性。

1.6 设计流程

本系统的设计遵循“自底向上”的模块化开发原则，分为五个主要阶段：

第一阶段：硬件选型与搭建，完成 MUSE Pi Pro 环境配置、传感器与执行器连线；

第二阶段：底层驱动开发，完成 I2C 库调用的 DHT11 读取及 SG90 脉冲控制逻辑；

第三阶段：云端 API 对接，集成百度 ASR/TTS 与通义千问大模型接口；

第四阶段：核心逻辑联调，实现语音识别文本与 AI 问诊结果的逻辑处理及硬件动作绑定；

第五阶段：系统封装与测试，将分立组件整合至实体纸盒模型中，并进行整体功能与稳定性测试。

第二部分 系统组成及功能说明

2.1 整体介绍

系统由核心控制层、语音交互层、云服务层和末端执行层四部分构成，整体框图如下：

- 核心控制层：以 MUSE Pi Pro 开发板为大脑，运行 Python 主程序，负责调度所有外设与网络请求。
- 语音交互层：通过 USB 麦克风采集用户语音，并通过 USB 音箱/耳机输出 TTS 合成语音。
- 云服务层：包括百度 AI 开放平台（提供 ASR 语音转文字、TTS 文字转语音）和阿里云通义千问大模型（提供医疗问诊与药品查询的智能文本生成）。
- 末端执行层：包括 DHT11 环境传感器、驱动药盒顶盖的 SG90 舵机，以及分布在 5 个药格中用于精准指引的 LED 指示灯。

系统运行时，用户语音经云端解析后返回结构化的药品方案；MUSE Pi Pro 解析该方案，若匹配到预设的绑定规则，立即输出 PWM 信号控制舵机动作，并输出高低电平控制对应药格的 LED 闪烁。

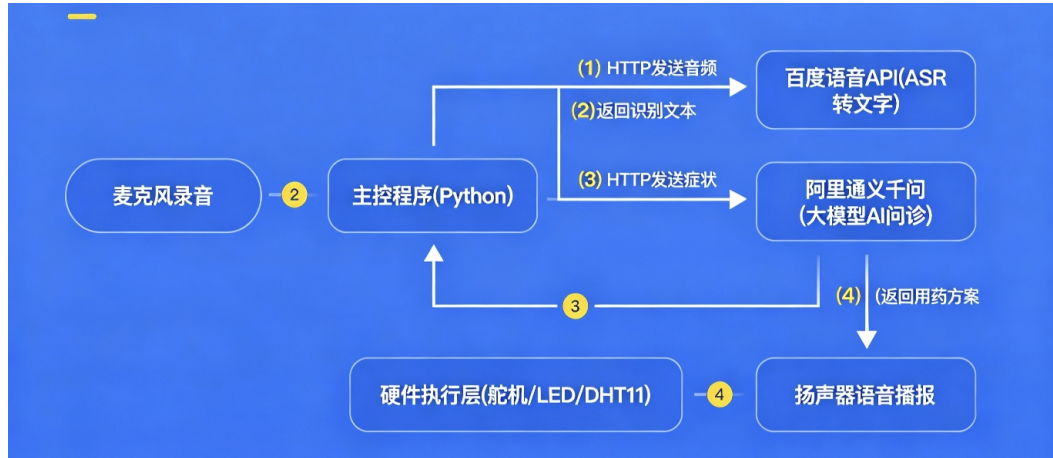


图 2.1 系统总体框架图

2.2 硬件系统介绍

系统的硬件选型以 MUSE Pi Pro 作为主控核心，充分利用其强大的 Linux 生态与多路 GPIO 控制能力。外围硬件通过面包板与杜邦线连接，包含 5 个蓝色 LED 指示模块、1 个 SG90 微型舵机、1 个 DHT11 温湿度传感器以及 USB 麦克风和扬声器。这些硬件组件被整合于一个定制化的多隔断纸质药盒模型内部，实现电路与机械结构的物理统一。

2.3 机械设计介绍

本系统的机械结构采用轻量化的 3d 打印模型进行封装。药盒内部通过隔板均分为 5 个独立的储物格，分别对应“晕车”、“关节痛”、“口腔溃疡”、“感冒发烧”和“腹泻”五种常用药类。

- 开合机构：利用 SG90 舵机作为动力源，安装在药盒外侧并连接纸板盖，通过舵机正反向旋转实现药盒顶盖的自动化开合。

- 指引机构：5 个蓝色发光二极管（LED）分别镶嵌在 5 个隔断的前方隔板上，用作药品位置指示。该设计成本低廉，易于加工，且在结构上直观展示了药盒分区取药的功能逻辑。

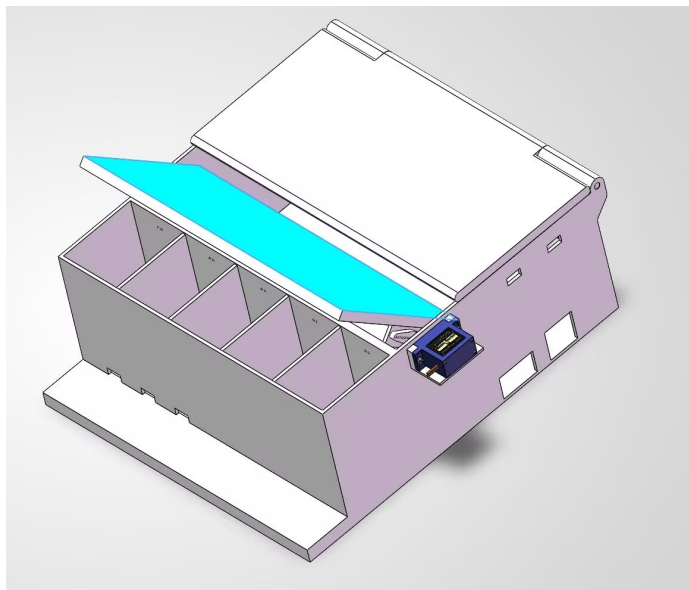


图 2.2 智能药盒 3d 模型

2.3.1 电路各模块介绍

系统电路以 MUSE Pi Pro 为核心，利用 lgpio 库完成 GPIO 控制，主要引脚分配及模块介绍如下：

核心主控模块（MUSE Pi Pro）：作为系统的大脑，负责运行 Python 主程序，处理音频数据的捕获，发起 HTTP 请求与云端（百度、阿里）通信，解析 JSON 返回数据，并根据解析结果控制 GPIO 引脚输出。



图 2.3 主控板

DHT11 温湿度传感器模块：连接至 MUSE Pi Pro 的 GPIO 37 引脚。采用单总线通信协议，用于采集环境温湿度。由于单总线通信易受干扰，代码中特别设计了最大 5 次的重试机制（max_retry=5），以确保读取数据的可靠性，并在读取失败时给出硬件上拉电阻的建议提示。

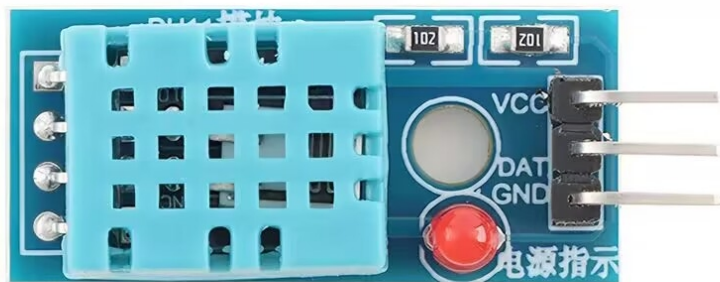


图 2.4 DHT11 温度传感器

SG90 舵机模块（药盒门驱动）： 连接至 MUSE Pi Pro 的 GPIO 40 引脚。工作周期为 20ms (50Hz)。代码中通过精确计算脉冲宽度 (0.5ms~2.5ms 对应 0°~180°) 实现角度控制。为了防止舵机在位置突变时产生抖动，设计了 servo_smooth_turn 函数，以 2° 为步进进行渐进式旋转，保证药盒盖开启过程中的平稳顺滑。



图 2.5 舵机

LED 指示灯模块： 系统使用了 5 个蓝色 LED 灯，引脚分别为 70, 71, 72, 73, 46。它们分别映射到 bind_list 中的 5 个不同症状。当系统判定用户症状匹配成功时，特定 LED 会以 0.5 秒 间隔闪烁 5 秒，直观提示用户从对应隔间取药。



图 2.6 LED 灯

音频采集与播放模块： 采用 USB 免驱麦克风连接至 MUSE Pi ProUSB 口，使用 sounddevice 库采集 16kHz、16bit 的 PCM 音频数据；音频输出则通过 3.5mm 耳机孔或 USB 音箱，利用 playsound 库播放百度 TTS 合成的 MP3 文件。



图 2.7 扬声器



图 2.8 麦克风

2.4 软件系统介绍

2.4.1 软件整体介绍

本系统的软件架构采用端云协同模式，运行在 MUSE Pi Pro 的 Linux 操作系统环境中，使用 Python 3 作为主要开发语言。整体系统主要分为 本地嵌入式控制层 和 云端 AI 服务层 两个逻辑部分：

1.本地嵌入式控制层：负责底层的硬件驱动、音频采集、用户交互菜单逻辑以及动作执行。它调用 lgpio 库实现 MUSE Pi Pro GPIO 引脚的精确控制，调用 sounddevice 库采集麦克风语音数据，并通过 requests 库发起网络请求。

2. 云端 AI 服务层：提供系统的“大脑”能力。系统通过 HTTP 协议与百度 AI 开放平台（实现语音转文字 ASR 及文字转语音 TTS）和阿里云通义千问大模型（实现智能问诊、药品查询、自由聊天）进行通信。

```
(demo-env) bianbu@spacebit-k1-x-MUSE-Pi-Pro-board: /asr-llm-tts/src$ python main_tts.py
===== 智能药盒语音交互系统 =====
===== 功能菜单 =====
1. 语音问诊 - 症状分析用药
2. 语音查询 - 药品百科
3. 语音自由聊天
4. 环境温度湿度检测
5. 修改
0. 退出程序
请输入序号(0/1/2/3/4/5):
```

图 2.9 功能菜单图

2.4.2 软件各模块介绍

根据代码结构，自顶向下可以分为 主控调度层、智能决策层、语音交互层、硬件驱动层 四个层级，具体设计如下：

主控调度层（main() 函数）

- 功能设计：系统的总入口。负责初始化硬件（hardware_init()），随后进入 while True 循环，提供功能菜单的文本交互界面（1.语音问诊、2.药品百科、3.自由聊天、4.温湿度检测、5.修改配置）。
- 关键控制流：根据用户输入的 choice 字符，分发至对应的功能分支。由于支持键盘中断（KeyboardInterrupt），系统在退出时会自动调用

hardware_release() 释放所有硬件引脚资源和删除临时音频文件。

- 输入/输出: 输入为终端键盘命令; 输出为终端打印信息及对应功能的函数调用。

智能决策层（大模型接口模块）本层是系统的“智慧大脑”，包含三个核心大模型 API 封装函数：

- ai_analyze_symptoms(symptom_text)（症状问诊）：
 - 设计流程：将固定的 5 种“症状→药品”映射规则拼接为 rule_str 写入 Prompt 提示词中。构建包含“严格遵守规则”、“固定输出 5 行格式”要求的 Prompt，使用 POST 请求发送给通义千问（qwen-turbo）。
 - 创新设计：在代码层面通过 Prompt Engineering，将硬编码规则（本地 5 个 LED 映射）与 AI 生成结果实现了约束对齐，确保 AI 给出的药品名能被后续代码解析。
- 关键输入/输出：输入为用户症状字符串（如“我晕车”）；输出为固定 5 行格式的用药方案（包含药品名称、剂量、次数、方式、注意事项）。
- ai_drug_intro(drug_name)（药品百科）：
 - 设计流程：将识别到的药品名称填入专门的 Prompt 模板（要求口语化、通俗易懂、适合老年人听），调用通义千问 API 获取详细介绍。
 - 关键输入/输出：输入为药品名称字符串；输出为适合语音播报的口语化药品介绍文案。
- ai_chat(user_text)（自由聊天）：
 - 设计流程：用于闲聊或简单的健康咨询，输入原始文本并限制大模型的角色为“温柔简洁的智能药盒助手”，以降低对话延迟。
 - 关键输入/输出：输入为用户输入的文本；输出为 AI 的简短口语化回复。

语音交互层（音频采集与合成模块）

- record_audio(prompt_msg) 与 speech_to_text(audio_data)：
 - 设计流程：使用 sounddevice 库进行 16kHz、16bit 单声道 PCM 录音，时长为 5 秒。录音结束后，将 numpy.int16 数组转为 bytes 二进制数据，通过调用百度语音的 client.asr() 接口进行云端语音识别。
 - 输入/输出：输入为 USB 麦克风音频；输出为识别结果字符串（如“晕车

) 。

- `text_to_speech_cn(text)` 与 `get_broadcast_text(full_text)`:
- 设计流程: 首先通过 `get_broadcast_text` 从 AI 生成的完整文本中截取有效播报内容。随后, 利用百度翻译 TTS 接口 (fanyi.baidu.com/gettts) 获取 MP3 音频数据, 并在本地保存为 `temp_tts.mp3`, 使用 `playsound` 库进行阻塞式播报, 播报完毕后自动删除临时文件以节省磁盘空间。
- 输入/输出: 输入为待播报的文本字符串; 输出为扬声器发出的语音提示。

硬件驱动层 (GPIO 控制模块) 本层基于 `Igpio` 库进行封装, 实现对底层硬件的直接控制: · `read_dht11()` 与 `get_dht11_data()` (DHT11 驱动):

- 设计流程: 采用单总线协议的精确时序控制。

主机拉低 20ms 发送起始信号。

拉高 20-40us, 随后 DHT11 响应发出 80us 低电平 + 80us 高电平。

循环 40 次读取位数据 (高电平持续时间 > 50us 代表 '1', 否则代表 '0') 。

数据验证: 将 40 位数据拆分为 5 个字节, 并校验前 4 个字节之和是否等于第 5 字节 (校验和)。不相等或超时则返回空, 并触发上层函数的 5 次重试机制。

- 关键输入/输出: 输入为 GPIO 37 引脚的电压变化; 输出为浮点型的温度 (`temp`) 和湿度 (`hum`) 变量。
- `servo_set_angle(angle)` 与 `servo_smooth_turn(target_angle)` (SG90 舵机驱动):
- 设计流程: 不依赖硬件的 PWM 波形生成, 而是利用 Python 的 `time.sleep()` 产生 软件 PWM 脉冲。计算目标角度所需的脉冲宽度 (0.5ms~2.5ms), 先拉高 GPIO 引脚, 延时对应脉宽, 再拉低引脚, 并等待剩余周期 (20ms - 脉宽) 以实现 50Hz 频率。
- 防抖设计: `servo_smooth_turn` 函数将目标角度与当前角度 `current_servo_angle` 对比, 以 2° 为步长进行渐进式转动, 并加入 `SERVO_STEP_DELAY` 延时, 极大减轻了舵机急停带来的机械抖动。
- 关键输入/输出: 输入为 0-180 的整数角度; 输出为 GPIO 40 引脚的高

低电平变化，驱动舵机转动。

- `led_set(pin, state)` 与 `led_blink_5s(target_pin)` (LED 指示灯驱动) :

- 设计流程：通过 `gpio.gpio_write` 直接控制指定引脚电平。

`led_blink_5s` 作为高亮指引函数，会在 5 秒内以 0.5秒间隔控制特定 LED 闪烁，直观引导用户找到对应药格。

- 输入/输出：输入为需要点亮的引脚号；输出为对应引脚的高低电平状态。(此处可插入一段核心代码逻辑的截图, 例如 `main()` 函数的主循环部分或者 `ai_analyze_symptoms` 的 API 调用部分，配合图注说明)

第三部分 完成情况及性能参数

3.1 整体介绍



图 3.1 智能药盒实物图

3.2 工程成果

3.2.1 机械成果

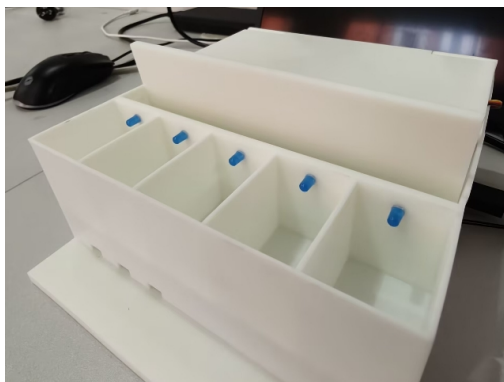


图 3.2 机械成果图

3.2.2 电路成果

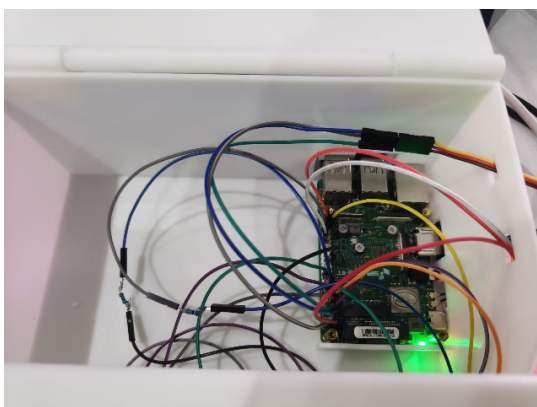


图 3.3 电路连接实物图

3.2.3 软件成果

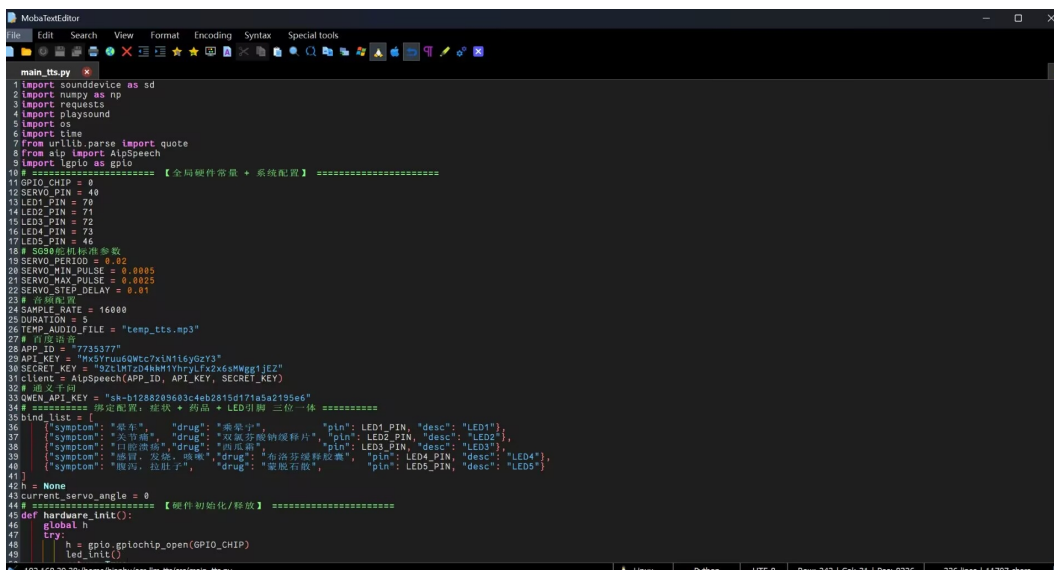


图 3.4 智能药盒运行代码图

```
GPIO CHIP = 0
DHT PIN = 37
SERVO PIN = 40
LED1 PIN = 70
LED2 PIN = 71
LED3 PIN = 72
LED4 PIN = 73
LED5 PIN = 46

# SG90舵机标准参数
SERVO PERIOD = 0.02
SERVO MIN PULSE = 0.0005
SERVO MAX PULSE = 0.0025
SERVO STEP DELAY = 0.01

# 音频配置
SAMPLE RATE = 16000
DURATION = 5
TEMP AUDIO FILE = "temp tts.mp3"

# 百度语音
APP ID = "7735377"
API KEY = "Mx5Yruu6QWtc7xiN1i6yGzY3"
SECRET KEY = "9ZtIMTzD4kkM1YhryLfx2x6sMWgg1jEZ"
client = AipSpeech(APP ID, API KEY, SECRET KEY)

# 通义千问
QWEN API KEY = "sk-b1288209603c4eb2815d171a5a2195e6"
```

图 3.5 全局硬件常量+系统配置

```
bind list = [
  {"symptom": "晕车", "drug": "乘晕宁", "pin": LED1 PIN, "desc": "LED1"},
  {"symptom": "关节痛", "drug": "双氯芬酸钠缓释片", "pin": LED2 PIN, "desc": "LED2"},
  {"symptom": "口腔溃疡", "drug": "西瓜霜", "pin": LED3 PIN, "desc": "LED3"},
  {"symptom": "感冒, 发烧, 咳嗽", "drug": "布洛芬缓释胶囊", "pin": LED4 PIN, "desc": "LED4"},
  {"symptom": "腹泻, 拉肚子", "drug": "蒙脱石散", "pin": LED5 PIN, "desc": "LED5"}
]
```

图 3.6 药品绑定配置

```
def ai analyze symptoms(symptom text):
    url = "https://dashscope.aliyuncs.com/api/v1/services/aigc/text-generation/generation"
    headers = {
        "Authorization": f"Bearer {QWEN_API_KEY}",
        "Content-Type": "application/json"
    }
    # 拼接当前所有 症状->药品 对应关系
    rule_str = ""
    for item in bind list:
        rule_str += f"{item['symptom']}→{item['drug']}, "
    rule_str = rule_str.rstrip(", ")
```

图 3.7 AI 问诊接口

第四部分 总结

4.1 可扩展之处

本作品已完成“端侧语音交互+云端 AI 大模型推理+本地硬件执行”的完整链路开发，并基于 3D 打印外壳实现了功能原型。面向实际产品化与功能深化，系统具有以下可扩展方向：

1. 药仓精细化管理：当前系统仅通过 LED 指示灯指引药格位置。未来可在药仓底部集成重力传感器或光电传感器，结合软件算法，实现“取药确认”与“漏取药”实时提醒，进一步保障老人用药安全。
2. 数据可视化与远程看护：可开发配套的微信小程序或 Web 端后台。一方面记录用户的历史问诊数据与用药习惯，另一方面通过 MQTT 协议将设备状态同步至家属端，实现远程健康守护。
3. 本地大模型优化与知识库扩充：由于 MUSE Pi Pro 具备 NPU 算力，后续可将通义千问云端模型替换为同等尺寸的本地量化模型（如 Qwen2.5-1.5B），实现完全离线的隐私问诊，并接入专业的《国家基本药物目录》数据库，提升用药建议的权威性。

交互维度升级：在现有“听”和“说”的基础上，引入触摸屏或表情识别摄像头，实现“多模态情绪感知”，使药盒能根据用户情绪状态给出更贴心的安慰性语音。

4.2 心得体会

通过本次智能药盒语音交互系统的设计与实现，我们收获了很多：

首先，我们深刻体会到了"学以致用"的含义。以前在课堂上学到的嵌入式系统、单片机、Python 编程等知识，这次都用到了实际项目中。从最初的需求分析、方案选型，到硬件连接、软件编程，再到调试测试、优化改进，整个过程走下来，我们对嵌入式系统开发有了更全面、更深入的理解。

其次，我们认识到了用户需求的重要性。在设计过程中，我们始终围绕"老年人好用、易用"这个核心目标，选择语音作为主要交互方式，尽量简化操作步骤，就是为了让老年用户不用学习就能上手。一个好的产品不应该是技术的堆砌，而应该真正解决用户的需求。

再次，我们学会了如何排查和解决问题。在调试过程中，我们遇到了很多问题，比如 DHT11 读取不稳定、舵机抖动、语音识别准确率不高等等。每次遇到问题，我们都会先分析可能的原因，然后逐一排查验证，最终找到解决办法。这个过程虽然有时候很煎熬，但解决问题后的成就感也是巨大的。

最后，我们也认识到了团队协作的重要性。项目开发过程中，我们分工合作，有人负责硬件，有人负责软件，有人负责测试，遇到问题一起讨论，大大提高了开发效率。同时我们也学会了如何清晰地表达自己的想法，如何倾听他人的意见，如何在有分歧时达成共识。

当然，我们的作品还有很多不足之处，比如外观还比较简陋，功能还不够丰富，用户体验还有提升空间。但通过这次比赛，我们学到了知识，锻炼了能力，也看到了自己的不足。这些收获比比赛结果更重要。

第五部分 参考文献

- [1] 进选时空 (SpacemiT). MUSE Pi Pro 开发板用户手册 [EB/OL]. (2024).
<https://www.spacemit.com/MUSE-pi-pro>.
- [2] 进选时空 (SpacemiT). K1 处理器技术数据手册 [EB/OL]. (2024).
<https://www.spacemit.com/k1-datasheet>.
- [3] Ollama. Ollama: Get up and running with large language models locally [EB/OL]. (2024). <https://ollama.com>.
- [4] Qwen Team. Qwen3: A new generation of large language models [EB/OL]. (2025).
<https://github.com/QwenLM/Qwen3>.
- [5] 百度 AI 开放平台. 百度语音识别与合成 API 文档 [EB/OL]. (2025).
<https://ai.baidu.com/tech/speech>.
- [6] sherpa-onnx. Next-generation speech processing toolkit [EB/OL]. (2025).
<https://github.com/k2-fsa/sherpa-onnx>.
- [7] 阿里云. 通义千问大模型 API 开发者文档 [EB/OL]. (2025).
<https://dashscope.aliyun.com/>.
- [8] lgpio. Local GPIO library for Linux. (2025). <https://github.com/joan2937/lg>.
- [9] Python Software Foundation. Python 3 Documentation [EB/OL]. (2025).
<https://docs.python.org/3/>.
- [10] AOSC. Debian Linux 系统管理手册 [EB/OL]. (2024). <https://www.debian.org/doc/>.
- [11] OpenCV Team. OpenCV 4.9.0 Documentation [EB/OL]. (2025).
<https://docs.opencv.org/>.
- [12] ALSA Project. Advanced Linux Sound Architecture Documentation [EB/OL]. (2025).
https://www.alsa-project.org/wiki/Main_Page.
- [13] libgpiod. GPIO character device library [EB/OL]. (2025).
<https://git.kernel.org/pub/scm/libs/libgpiod/libgpiod.git/>.
- [14] 工业和信息化部. 智慧健康养老产业发展行动计划 (2021-2025 年)
)
[EB/OL]. (2021). https://www.gov.cn/zhengce/zhengceku/2021-10/22/content_5644316.htm